PARALLEL PROGRAMMING

Remote execution, Embarrassingly parallel program,
Basics using MPI (in C/C++)
Lecture 4&5

MOTIVATION

Utilize many computers or cores
Make the program faster
Working with larger problem size
Insight into computer communication mechanism

REMOTE SHELL

- What is needed?
  - Address of the remote server
  - Account in the remote server
  - Sufficient connection speed
  - Remote shell client: ssh client, putty, winscp
- Using secure shell (ssh)
  - Syntax: `ssh user@server.somewhere`

AUTHENTICATION

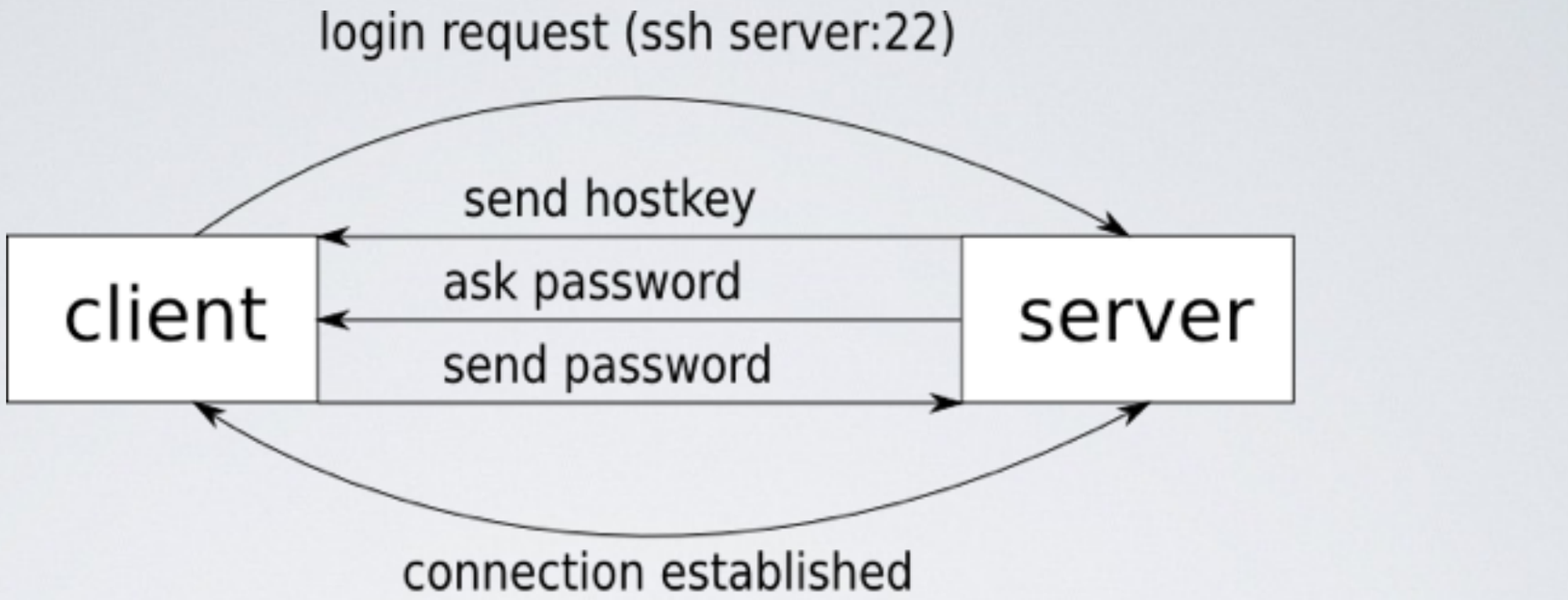
- Password authentication
  - A user is asked for password every time he logins.
- Key authentication
  - No password required
  - A pair of keys required:
    private key (on client), public key on server

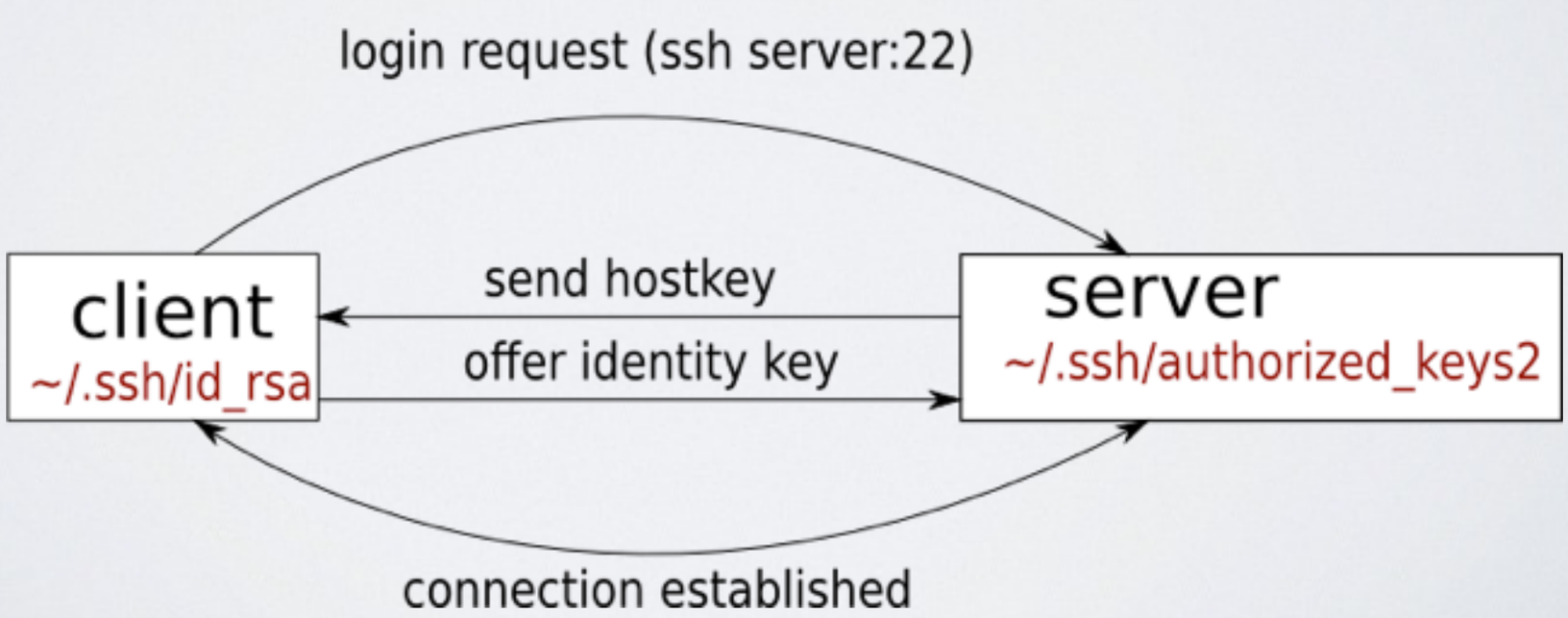
AUTHENTICATION

- Key authentication is saver!
- No need to type password on login (**password-less**)
- Creating ssh key-pair: ssh-keygen
  One may put passphrase to unlock the key for more security
  Key default location: `.ssh/id_rsa .ssh/id_rsa.pub`
- Keep private key secret, transfer/distribute only public key
- A key is not valid without a correct pair

Never send private key via email or put in public storage!

AUTHENTICATION MECHANISM





SSH FILES

- id_rsa, private key, stored at client side
- id_rsa.pub, public key, distributed to all server you want to access
- known_hosts, list of the known hosts. If the host key in server side changed, this file must be updated
- authorized_keys (or authorized_keys2), the list of public keys of to whom the access granted (server side)
- config, local user configuration file (client side)

All these files are stored in $HOME/.ssh

CONFIG FILE EXAMPLE

```
Host clus
    Hostname some.where.de
    ForwardX11 yes
    Port 43523
    IdentityFile ~/.ssh/keys/mykey
    User myname
```

Using this configuration file making connection is easier:

```
$ ssh clus
```

more information: **man ssh_config**

RUNNING REMOTE PROGRAM

- Syntax
  - foreground

    ```
    ssh user@remote.server /program/path
    ```

  - background

    ```
    ssh user@remote.server /program/path &
    ```

- Example

  ```
  ssh clus '
  cd ~/Programs/lmp/test/al-heating
  $HOME/bin/lmp_rzn < in.alslab' &
  ```

EMBARRASSINGLY PARALLEL PROGRAM

DEFINITION

- Running many independent programs parallel in many computers/cores

- Data is transferred and communicated via a normal data storage (hard-disk, temporary files)

STRATEGY

REQUIREMENTS

- List of usable computer address
  - List of computer (**node**) address/name in a "hostfile"
    - On a multicore computer: fork process, or use "localhost"
- Communication:
  program initialization, input, output, program identity, etc.
- Hand shake: signals between remote processes
- Shared storage space, password-less ssh connection

MAKING A LIST OF COMPUTER NAMES

- If you know the computer to use, write it in a text file (e.g. hostfile) one node per line. Put the same name many times if the node has many cores.

```
node01-0001
node01-0002
```

- Host name: used as part of program identity
- To count the number of core used: `echo hostfile|wc -l`
- The hostname is used to execute remote program using ssh

```
for h in $(cat hostfile); do
  ssh $h /program/path &
done
# wait until all program finished
```

COMMUNICATION

- All program can access the a common location on a shared filesystem

- File can be used as a communication slot
  → eg. use information (host, process id, etc) as filename

- Input files maybe prepared prior to calling remote programs

- Give each process an ID: Can be the PID or manually assigned number

- All process communicate using this ID

EXAMPLE

```
#!/bin/bash

PRGPATH=$0
MYID=$1
HOST=$2

# empty $MYID  means root process
if [[ -z $MYID ]]; then
  MYID=0
  PRGPATH=$PWD/$(basename $0)
fi

die() {
  echo $@ >&2
  exit 1
}
```

```
##### process spawning #####
if [[ $MYID == 0 ]]; then
  [[ -f hostfile ]]||
  die "can not find hostfile"
  # first line is root
  CID=0
  for h in $(cat hostfile); do
     if [[ $CID == 0 ]]; then
       HOST=$h
     else
       ssh $h $PRGPATH $CID $h &
     fi
     CID=$(( $CID + 1 ))
  done
fi
################################
```

EXAMPLE

```
###### parallel program body ######

cd $(dirname $PRGPATH)
NHOST=$(cat hostfile|wc -l)
MYSLOT=runid.$MYID.$HOST
echo running > $MYSLOT

# do something in all processes

for ((a=1;a<100;a++)); do
  echo $a
  sleep 0.5
done >> $MYSLOT

echo finished >> $MYSLOT

#######################################
```

IO PROBLEM

- Reading usually does not have any problem
- Writing to a same file: raise a racing problem
  - Use file locking or semaphore
  - Use different file, combine by root process at the end